ANÁLISIS DE ENRIQUECIMIENTO DE GENES (I)

BIOINFORMÁTICA

- EPD 6 -

Luis Carmona Berdugo

Problema 1

Realice un análisis completo de enriquecimiento de genes con las aplicaciones g:Profiler y EnrichR de su microarray NCBI. Identifique y comente los términos GO más relevantes. Para ello, debe realizar previamente una limpieza, preprocesamiento y clusterización del dataset si es necesario.

Voy a usar el microarray “GSE27272_non-normalized_cord_blood” de la serie “GSE27272”, el cual hace referencia a un estudio exhaustivo de las alteraciones del transcriptoma relacionadas con el humo del tabaco en células maternas y fetales. Más en concreto a los datos correspondientes a la sangre.

En primer lugar comenzaré leyendo los datos de mi MicroArray y almacenando estos en df.

```
R Console
> # Cargar librerias necesarias
> library(data.table)  # Para carga eficiente de datos grandes
> library(tidyverse)   # Para manipulación de datos
Error en library(tidyverse): no hay paquete llamado 'tidyverse'
> library(cluster)     # Para clustering
> library(factoextra)  # Para visualización
>
> # Definir rutas de archivo
> input_file <- "C:/Users/luisc/Desktop/Cosas/Universidad/3º Tercero/Segundo Cuá
> output_file <- "C:/Users/luisc/Desktop/MicroArray_Procesado.txt"
>
> # Cargar los datos
> df <- fread(input_file, header = TRUE, sep = "\t", na.strings = c("", "NA"))
>
> # Revisar estructura
> print(head(df))
        V1      302D      303D      305D      306D      307D      308D
   <lgcl>    <num>     <num>     <num>     <num>     <num>     <num>
1:     NA 1518.24939 1303.37385 1103.28450 527.41778 1066.03891 866.02560
2:     NA   56.84939   32.47547   58.08450  13.45980   49.23891 123.62560
3:     NA   32.15110   32.07581   26.78540  16.90791   31.93897  16.90419
4:     NA  146.44939  163.17385  151.18450  68.11778  105.63891 145.62560
5:     NA 1295.04939 1114.47385  947.88450 411.81778 1298.23891 972.32560
6:     NA   16.30173   12.43397   10.25056  12.75662   12.91881  10.92337
       311D      312D      313D      314D      315D      316D      317D
     <num>     <num>     <num>     <num>     <num>     <num>     <num>
1: 530.02042 448.997641 479.83030 396.634720 411.704012 782.98728 579.43331
2:  76.42042  53.297641  27.73030  44.034720  30.504012  54.88728  88.13331
3:  12.00207  13.253549  16.34727   7.870625   6.420793  23.78758  12.72304
4:  38.42042  54.697641  54.13030  49.334720  44.704012  53.28728  68.13331
5: 406.02042 330.597641 338.23030 366.234720 281.104012 350.88728 449.93331
6:  13.22215   8.898266  13.41543   6.663470  11.277033  11.50871  12.63080
       318D      320D      322D      324D      327D      328D      330D
     <num>     <num>     <num>     <num>     <num>     <num>     <num>
1: 724.16303 562.781430 1014.908875 711.89286 440.76427 595.49760 446.78986
2: 187.06303  43.081430  110.708875  78.39286  24.26591  28.89795  49.38986
3:  20.86372   7.645312    8.669432  11.41081  18.30392  12.35420  16.88775
4:  66.56303  49.081430  108.608875  50.89286  37.86427  43.89760  71.38986
5: 607.86303 341.181430  864.908875 326.69286 281.56427 335.89760 430.88986
6:  21.96332  13.448789   10.421542  10.97236  10.90178  15.24532  11.49636
       331D      332D      334D      335D      336D      337D      338D
     <num>     <num>     <num>     <num>     <num>     <num>     <num>
1: 601.54117 946.79403 672.73646 755.76187 1343.11901 415.928022 944.440390
```

```
C:\Users\luisc\Desktop\Cosas\Universidad\3º Tercero\Segundo Cuatrismestre\Bioinformática\R\EPD6P1.R - Editor R
# Cargar librerias necesarias
library(data.table)  # Para carga eficiente de datos grandes
library(cluster)     # Para clustering
library(factoextra)  # Para visualización

# Definir rutas de archivo
input_file <- "C:/Users/luisc/Desktop/Cosas/Universidad/3º Tercero/Segundo Cuatrismestre/Bioinformática/Expander/Mic
output_file <- "C:/Users/luisc/Desktop/MicroArray_Procesado.txt"

# Cargar los datos
df <- fread(input_file, header = TRUE, sep = "\t", na.strings = c("", "NA"))

# Revisar estructura
print(head(df))
```

Tras esto procederé a la limpieza de los datos.

```
>
> # ------- LIMPIEZA DE DATOS --------
>
> # Eliminar la primera columna si solo contiene IDs o valores vacios
> if (all(is.na(df[[1]]))) {
+   df <- df[, -1, with = FALSE]
+ }
>
> # Asegurar que todas las columnas sean numéricas
> numeric_cols <- names(df)[sapply(df, is.character)]
> df[, (numeric_cols) := lapply(.SD, as.numeric), .SDcols = numeric_cols]
>
> # Eliminar columnas con más del 30% de valores NA
> threshold <- 0.3 * nrow(df)
> df <- df[, which(colSums(is.na(df)) < threshold), with = FALSE]
>
> # Eliminar filas con valores NA restantes
> df <- na.omit(df)
>
> # -------- NORMALIZACIÓN --------
>
```

```
# Revisar estructura
print(head(df))

# ------- LIMPIEZA DE DATOS --------

# Eliminar la primera columna si solo contiene IDs o valores vacios
if (all(is.na(df[[1]]))) {
  df <- df[, -1, with = FALSE]
}

# Asegurar que todas las columnas sean numéricas
numeric_cols <- names(df)[sapply(df, is.character)]
df[, (numeric_cols) := lapply(.SD, as.numeric), .SDcols = numeric_cols]

# Eliminar columnas con más del 30% de valores NA
threshold <- 0.3 * nrow(df)
df <- df[, which(colSums(is.na(df)) < threshold), with = FALSE]

# Eliminar filas con valores NA restantes
df <- na.omit(df)
```

Para el preprocesamiento usaré el método de normalización.

```
>
> # -------- NORMALIZACIÓN --------
>
> # Normalizar los datos (Z-score)
> df_scaled <- scale(df)
```

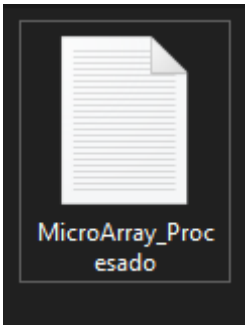
```
# -------- NORMALIZACIÓN --------

# Normalizar los datos (Z-score)
df_scaled <- scale(df)
```

Finalmente le haré un clustering y lo almacenaré en otro archivo para su posterior análisis.

```
> # -------- CLUSTERING (si es necesario) --------
>
> # Determinar número óptimo de clusters con el método del codo
> fviz_nbclust(df_scaled, kmeans, method = "wss")
>
> # Aplicar k-means clustering con el número óptimo de clusters (ejemplo: k=3)
> set.seed(123)
> kmeans_result <- kmeans(df_scaled, centers = 3, nstart = 25)
>
> # Agregar el cluster al dataset
> df$Cluster <- factor(kmeans_result$cluster)
>
> # Guardar los datos procesados en la ruta especificada
> write.table(df, output_file, sep = "\t", row.names = FALSE, quote = FALSE)
>
> # Mensaje de éxito
> cat("✅ Archivo guardado en:", output_file, "\n")
✅ Archivo guardado en: C:/Users/luisc/Desktop/MicroArray_Procesado.txt
>
> |
```

```
# Normalizar los datos (Z-score)
df_scaled <- scale(df)

# -------- CLUSTERING (si es necesario) --------

# Determinar número óptimo de clusters con el método del codo
fviz_nbclust(df_scaled, kmeans, method = "wss")

# Aplicar k-means clustering con el número óptimo de clusters (ejemplo: k=3)
set.seed(123)
kmeans_result <- kmeans(df_scaled, centers = 3, nstart = 25)

# Agregar el cluster al dataset
df$Cluster <- factor(kmeans_result$cluster)

# Guardar los datos procesados en la ruta especificada
write.table(df, output_file, sep = "\t", row.names = FALSE, quote = FALSE)

# Mensaje de éxito
cat("✅Archivo guardado en:", output_file, "\n")
```

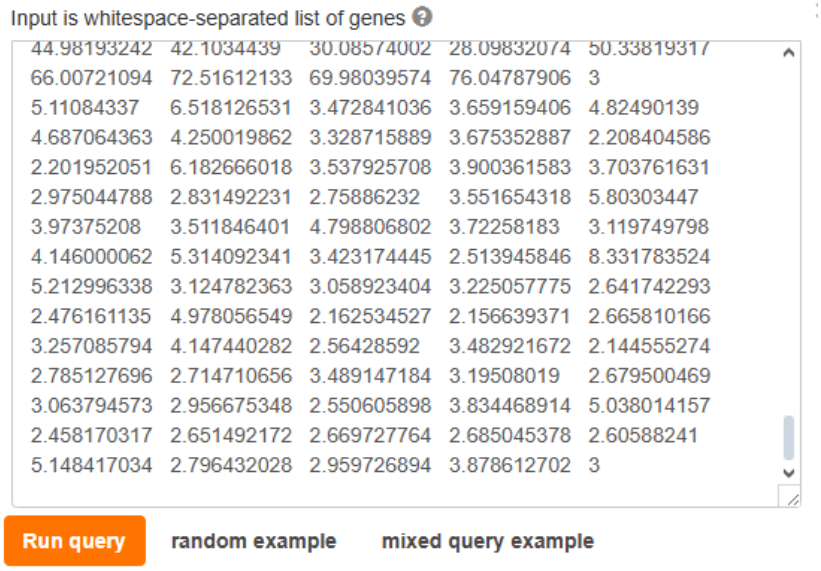
Una vez terminado el proceso deberemos tener el archivo procesado en la ruta correspondiente, en mi caso el Escritorio.



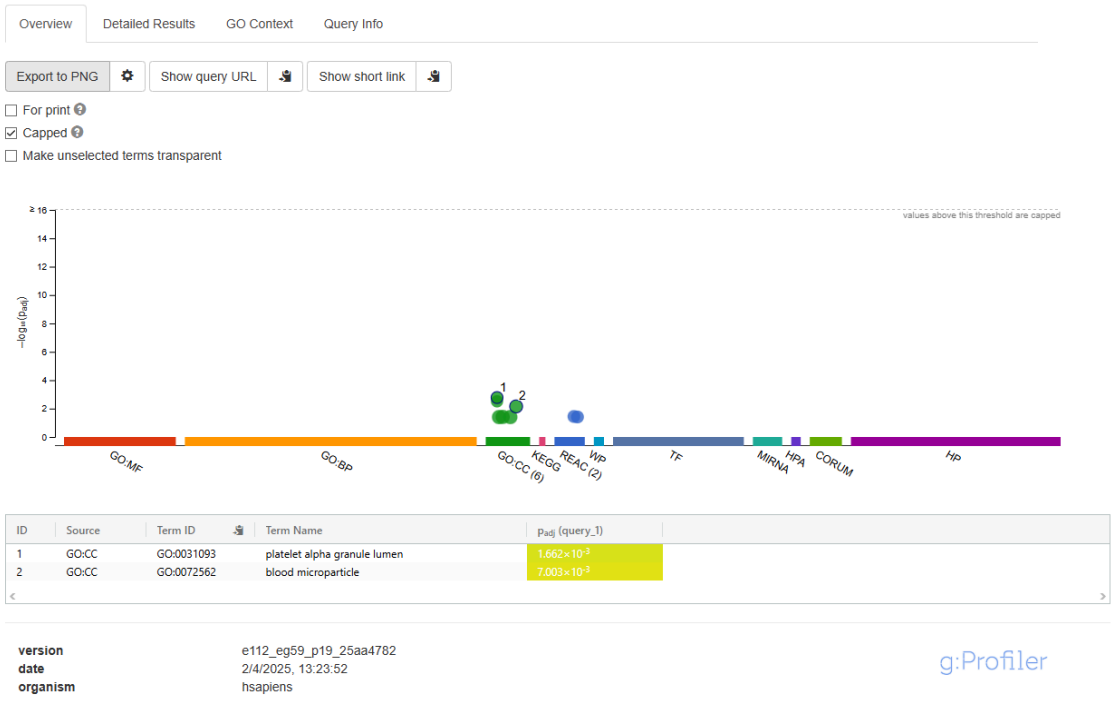
Tras esto pasaremos a hacer un análisis completo en ambas aplicaciones g:Profiler y EnrichR del MicroArray procesado.

g:Profiler

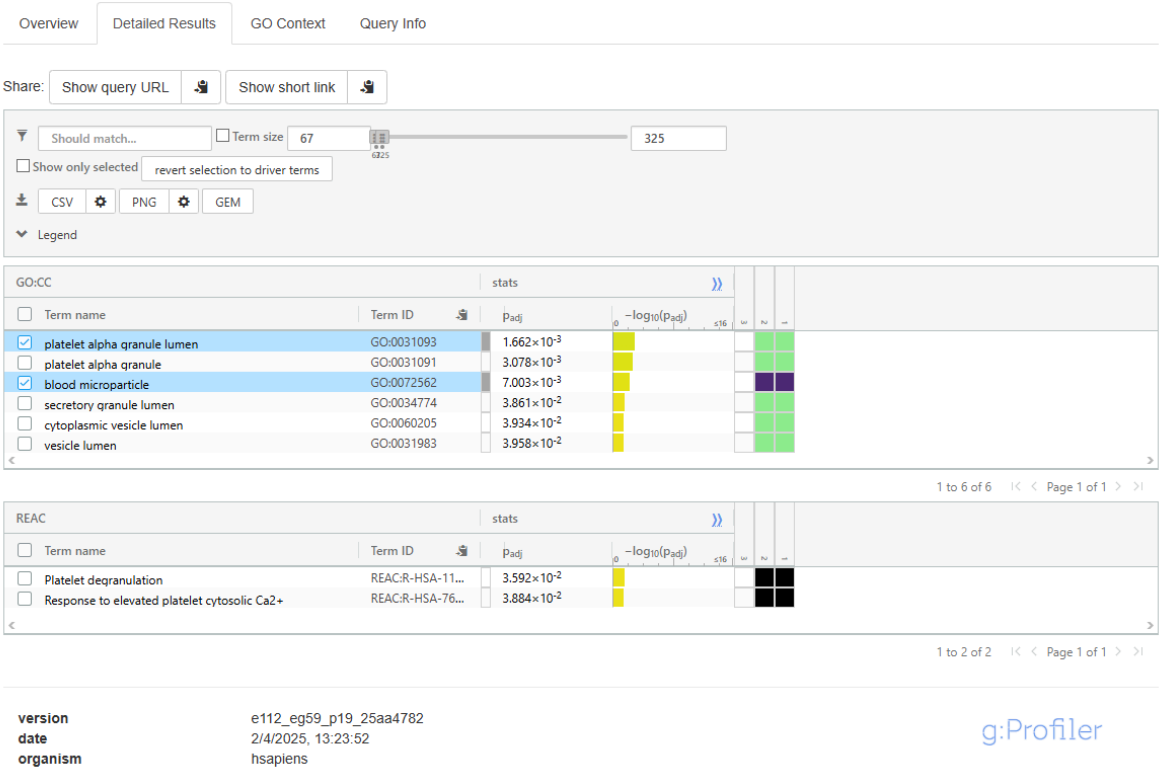
Subimos el archivo a la web y pulsamos sobre Run Query.



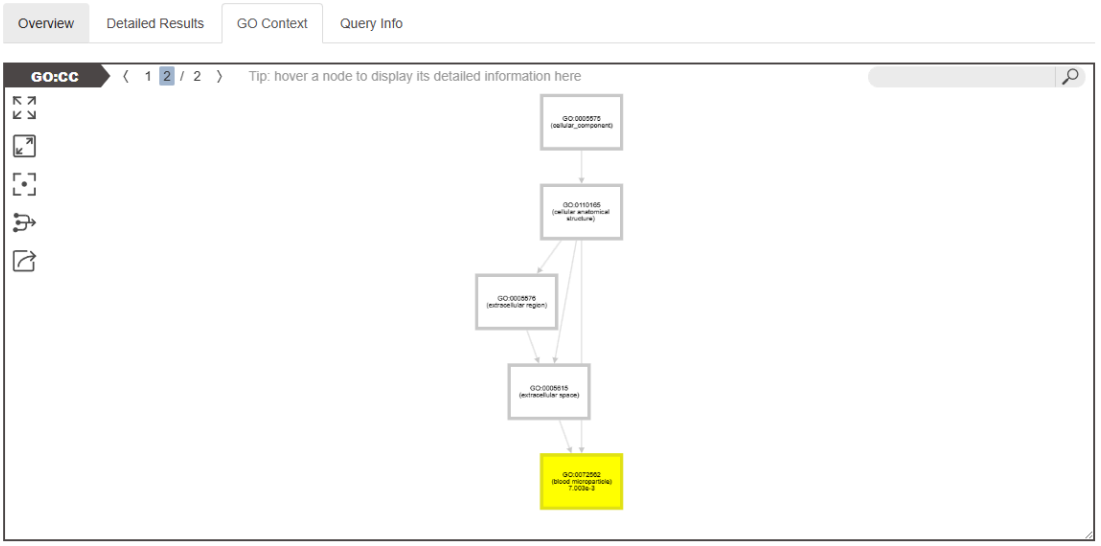
Tras esto debemos esperar a que termine el proceso, una vez finalizado podremos ver los resultados. En primer lugar vemos la pestaña Overview.



Podemos sacar ciertas conclusiones, como que el término "platelet alpha granule lumen" destaca por su significación estadística y especificidad funcional, vinculado a procesos precisos en plaquetas. En comparación, "blood microparticle", más general, es significativo pero menos detallado para interpretar mecanismos específicos en el contexto experimental.
Tras esto vemos la pestaña Detailed Results.



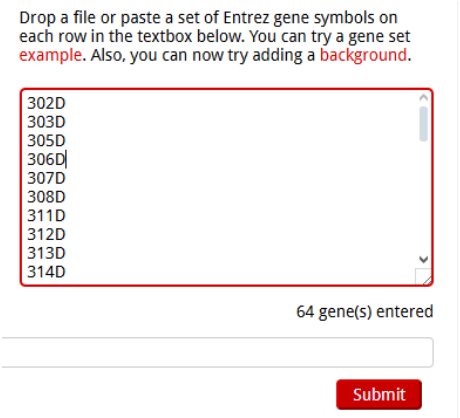
Las conclusiones que podemos sacar de aquí podrían ser que destacan "platelet alpha granule lumen" y "platelet alpha granule" por su alta significación estadística y especificidad en procesos plaquetarios, mientras que "blood microparticle" resalta por su relevancia en coagulación e inmunidad, apoyando la hipótesis experimental.
Por último en la pestaña GO Context podemos ver lo siguiente:



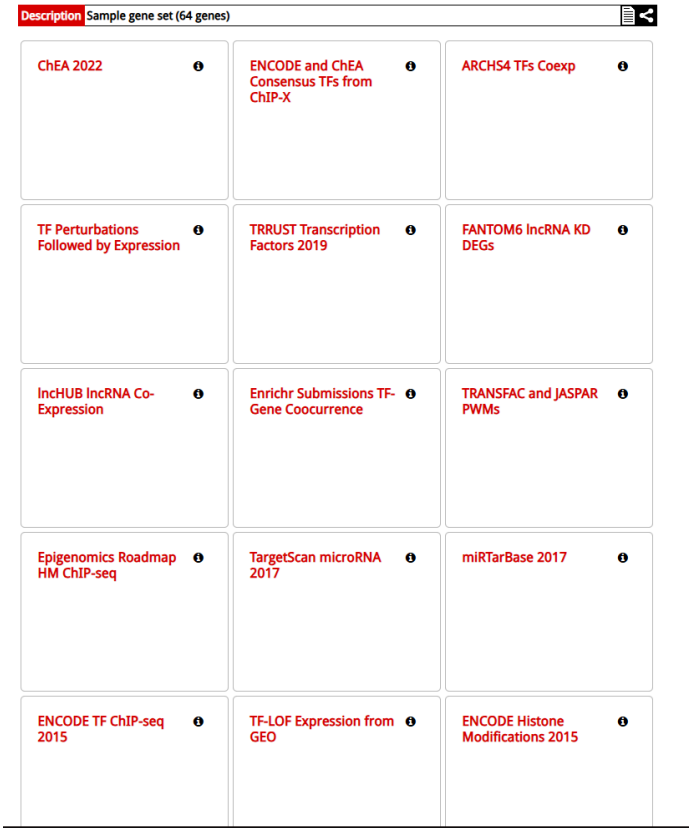
Llegamos a la conclusión de que es un diagrama jerárquico que muestra la progresión desde términos generales como cellular_component hacia categorías más específicas. La relevancia de blood microparticle, resaltada en amarillo, destaca su especialización en señalización y coagulación, facilitando interpretaciones detalladas.

EnrichR

Para finalizar subimos el archivo procesado también a esta herramienta y pulsamos el botón de Submit.



Tras terminar el proceso la herramienta nos lanza el siguiente resultado.



El análisis muestra varias herramientas de conjuntos de genes en Enrichr, como ChEA, ENCODE, ARCHS4, TRRUST, TargetScan y miRTarBase. Estas evalúan factores de transcripción, modificaciones epigenéticas y microARNs, facilitando la investigación en regulación genética y expresión génica.

Problema 2

Realice un análisis completo de enriquecimiento de genes con las aplicaciones GeneOntology y GOrilla de su dataset RNA-Seq de NCBI. Identifique y comente los términos GO más relevantes. Para ello, debe realizar previamente una limpieza, preprocesamiento y clusterización del dataset si es necesario.

Voy a usar el microarray “GSE100797_ProcessedData” de la serie “GSE100797”, el cual hace referencia a la carga mutacional y neoantigénica predicen el beneficio clínico de la terapia con células T adoptivas en el melanoma.

En primer lugar comenzaré leyendo los datos de mi dataset y almacenando estos en dataGDS.

```
R Console
> # Cargar librerias necesarias
> library(caret)       # Preprocesamiento de datos
> library(cluster)     # Algoritmos de clusterización
> library(factoextra)  # Visualización de clustering
> library(data.table)  # Manejo eficiente de datos
>
> # Definir la ruta del archivo
> ruta_archivo <- "C:/Users/luisc/Desktop/Cosas/Universidad/3º Tercero/Segundo Cuatr$
>
> # Cargar los datos
> dataGDS <- fread(ruta_archivo, header = TRUE, sep = "\t")
>
> # Ver las primeras filas para inspección
> head(dataGDS)
       V1 MM909_01_1 MM909_02_1 MM909_03_1 MM909_06_1 MM909_11_1 MM909_14_1
    <char>      <num>      <num>      <num>      <num>      <num>      <num>
1:    A1BG    1.98900    1.58027    2.10649    3.29450    1.58371    1.96644
2:    A1CF    0.00000    0.00000    0.00000    0.00000    0.00000    0.00000
3:     A2M    7.87575   10.00009    5.55774    6.87104    8.38983   10.09758
4:   A2ML1    0.00000    0.00000    0.14526    0.04093    0.06347    0.06982
5: A3GALT2    0.00000    0.00000    0.14072    0.00000    0.00000    0.00000
6:  A4GALT    1.69386    0.54076    0.95374    2.23522    1.36743    1.76861
   MM909_15_2 MM909_16_1 MM909_17_1 MM909_22_2 MM909_25_1 MM909_26_1 MM909_27_1
        <num>      <num>      <num>      <num>      <num>      <num>      <num>
1:    4.13673    2.74361    3.67576    1.59783    3.22909    1.41953    1.66138
2:    0.00000    0.00000    0.00000    0.00000    0.00869    0.00000    0.00000
3:    7.41774    5.32536    6.14495    8.74414    8.54103    6.96894    5.92766
4:    0.02834    0.04180    0.06613    0.08091    0.00000    0.01511    0.03795
5:    0.00000    0.00000    0.06193    0.00000    0.14349    0.00000    0.07993
6:    1.16800    0.85947    1.09778    4.09500    2.57049    1.77463    0.87132
   MM909_29_1 MM909_31_1 MM909_34_2 MM909_35_2 MM909_36_1 MM909_37_1 MM909_40_1
        <num>      <num>      <num>      <num>      <num>      <num>      <num>
1:    0.84260    1.60720    4.00498    1.33098    2.60470    1.04829    1.26327
2:    0.00000    0.00000    0.01034    0.00000    0.00000    0.00000    0.00000
3:    8.12748    7.39345    7.73267   10.16770    7.23054    8.22219    8.56705
4:    0.00225    0.00000    0.04266    0.00000    0.00000    0.00000    0.01303
5:    0.00000    0.02035    0.21953    0.05331    0.00000    0.00000    0.00000
6:    1.83496    0.59895    0.67195    1.64446    0.88232    2.43339    0.73737
   MM909_42_1 MM909_43_1 MM909_45_1 MM909_46_1 MM909_47_1
        <num>      <num>      <num>      <num>      <num>
1:    1.12852    1.56986    0.82416    1.55491    1.75340
2:    0.00781    0.00000    0.00000    0.00000    0.01073
3:    9.51885    6.87358    7.92530    8.34944    9.87299
4:    0.00000    0.04716    0.00584    0.00937    0.00000
5:    0.00000    0.00000    0.00000    0.00000    0.05914
6:    1.02665    1.07589    2.69675    1.27845    0.59805
>
```

```
C:\Users\luisc\Desktop\Cosas\Universidad\3º Tercero\Segundo Cuatrimestre\Bioinformática\R\EP...
# Cargar librerias necesarias
library(caret)       # Preprocesamiento de datos
library(cluster)     # Algoritmos de clusterización
library(factoextra)  # Visualización de clustering
library(data.table)  # Manejo eficiente de datos

# Definir la ruta del archivo
ruta_archivo <- "C:/Users/luisc/Desktop/Cosas/Universidad/3º Tercero/Segundo Cua

# Cargar los datos
dataGDS <- fread(ruta_archivo, header = TRUE, sep = "\t")

# Ver las primeras filas para inspección
head(dataGDS)
```

Tras esto procederé a la limpieza de los datos.

```
> # --- LIMPIEZA DE DATOS ---
> # Eliminar valores nulos
> dataGDS <- na.omit(dataGDS)
>
> # Identificar y reemplazar valores atípicos por la media de la columna
> samplesGDS <- colnames(dataGDS)[-1]  # Excluir primera columna si es identificador$
>
> for (col in samplesGDS) {
+   Q1 <- quantile(dataGDS[[col]], 0.25, na.rm = TRUE)
+   Q3 <- quantile(dataGDS[[col]], 0.75, na.rm = TRUE)
+   IQR <- Q3 - Q1
+   limitDown <- Q1 - 1.5 * IQR
+   limitUp <- Q3 + 1.5 * IQR
+   media <- mean(dataGDS[[col]], na.rm = TRUE)
+
+   dataGDS[[col]][dataGDS[[col]] >= limitUp] <- media
+   dataGDS[[col]][dataGDS[[col]] <= limitDown] <- media
+ }
```

```
# --- LIMPIEZA DE DATOS ---
# Eliminar valores nulos
dataGDS <- na.omit(dataGDS)

# Identificar y reemplazar valores atípicos por la media de la columna
samplesGDS <- colnames(dataGDS)[-1]  # Excluir primera columna si es identificad

for (col in samplesGDS) {
  Q1 <- quantile(dataGDS[[col]], 0.25, na.rm = TRUE)
  Q3 <- quantile(dataGDS[[col]], 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1
  limitDown <- Q1 - 1.5 * IQR
  limitUp <- Q3 + 1.5 * IQR
  media <- mean(dataGDS[[col]], na.rm = TRUE)

  dataGDS[[col]][dataGDS[[col]] >= limitUp] <- media
  dataGDS[[col]][dataGDS[[col]] <= limitDown] <- media
```

Para continuar llevaré a cabo el preprocesamiento.

```
> # --- PREPROCESAMIENTO ---
> # Preprocesamiento: centrar y escalar los datos
> preprocessParams <- preProcess(dataGDS[, -1, with = FALSE], method = c("center", "$
>
> # Convertir a data.frame antes de la asignación para evitar problemas de compatibi$
> dataGDS_scaled <- as.data.frame(predict(preprocessParams, dataGDS[, -1, with = FAL$
>
> # Reintegrar la columna de identificadores (asumiendo que la primera columna es id$
> dataGDS <- cbind(dataGDS[, 1, with = FALSE], dataGDS_scaled)
>
> # Verificar estructura final
> str(dataGDS)
Classes 'data.table' and 'data.frame':  18418 obs. of  26 variables:
 $ V1        : chr  "A1BG" "A1CF" "A2M" "A2ML1" ...
 $ MM909_01_1: num  -0.13 -1.09 2.71 -1.09 -1.09 ...
 $ MM909_02_1: num  -0.3269 -1.0922 0.0308 -1.0922 -1.0922 ...
 $ MM909_03_1: num  -0.0654 -1.0747 1.5881 -1.0051 -1.0073 ...
 $ MM909_06_1: num  0.503 -1.096 2.239 -1.076 -1.096 ...
 $ MM909_11_1: num  -0.331 -1.094 2.947 -1.063 -1.094 ...
 $ MM909_14_1: num  -0.1391 -1.0873 0.0284 -1.0537 -1.0873 ...
 $ MM909_15_2: num  0.926 -1.08 2.517 -1.066 -1.08 ...
 $ MM909_16_1: num  0.23 -1.08 1.47 -1.06 -1.08 ...
 $ MM909_17_1: num  0.703 -1.079 1.901 -1.047 -1.049 ...
 $ MM909_22_2: num  -0.329 -1.102 3.131 -1.063 -1.102 ...
 $ MM909_25_1: num  0.464 -1.092 3.031 -1.096 -1.027 ...
 $ MM909_26_1: num  -0.413 -1.097 2.261 -1.09 -1.097 ...
 $ MM909_27_1: num  -0.28 -1.07 1.76 -1.06 -1.04 ...
 $ MM909_29_1: num  -0.682 -1.092 2.866 -1.091 -1.092 ...
 $ MM909_31_1: num  -0.311 -1.089 2.488 -1.089 -1.079 ...
 $ MM909_34_2: num  0.86 -1.075 2.666 -1.059 -0.973 ...
 $ MM909_35_2: num  -0.456 -1.0974 0.0298 -1.0974 -1.0717 ...
 $ MM909_36_1: num  0.177 -1.088 2.423 -1.088 -1.088 ...
 $ MM909_37_1: num  -0.596 -1.103 2.878 -1.103 -1.103 ...
 $ MM909_40_1: num  -0.486 -1.096 3.042 -1.09 -1.096 ...
 $ MM909_42_1: num  -0.545 -1.085 0.025 -1.089 -1.089 ...
 $ MM909_43_1: num  -0.337 -1.093 2.218 -1.07 -1.093 ...
 $ MM909_45_1: num  -0.7 -1.1 2.75 -1.1 -1.1 ...
 $ MM909_46_1: num  -0.342 -1.095 2.947 -1.09 -1.095 ...
 $ MM909_47_1: num  -0.2351 -1.0751 0.0279 -1.0803 -1.0518 ...
 - attr(*, ".internal.selfref")=<externalptr>
>
```

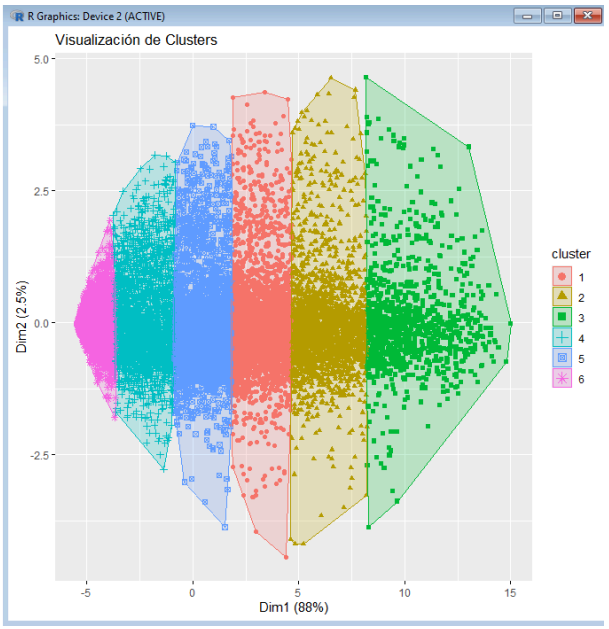
```
C:\Users\luisc\Desktop\Cosas\Universidad\3º Tercero\Segundo Cuatrismestre\Bioinformática\R\EPD6P2.R - Editor R

for (col in samplesGDS) {
  Q1 <- quantile(dataGDS[[col]], 0.25, na.rm = TRUE)
  Q3 <- quantile(dataGDS[[col]], 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1
  limitDown <- Q1 - 1.5 * IQR
  limitUp <- Q3 + 1.5 * IQR
  media <- mean(dataGDS[[col]], na.rm = TRUE)

  dataGDS[[col]][dataGDS[[col]] >= limitUp] <- media
  dataGDS[[col]][dataGDS[[col]] <= limitDown] <- media
}

# --- PREPROCESAMIENTO ---
# Preprocesamiento: centrar y escalar los datos
preprocessParams <- preProcess(dataGDS[, -1, with = FALSE], method = c("center", "scale"))

# Convertir a data.frame antes de la asignación para evitar problemas de compatibilidad
dataGDS_scaled <- as.data.frame(predict(preprocessParams, dataGDS[, -1, with = FALSE]))

# Reintegrar la columna de identificadores (asumiendo que la primera columna es identificador)
dataGDS <- cbind(dataGDS[, 1, with = FALSE], dataGDS_scaled)

# Verificar estructura final
str(dataGDS)
```

Finalmente le haré un clustering y lo almacenaré en otro archivo para su posterior análisis.

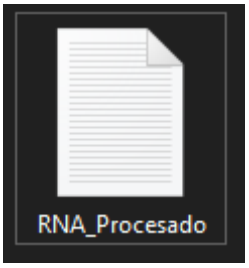
```
R Console

> # --- CLUSTERIZACIÓN ---
> # Determinar el número óptimo de clusters con el método del codo
> wss <- numeric(15)
> for (i in 1:15) {
+   kmeans_result <- kmeans(dataGDS[, -1, with = FALSE], centers = i, nstart = 10)
+   wss[i] <- kmeans_result$tot.withinss
+ }
Hubo 22 avisos (use warnings() para verlos)
>
> # Graficar la selección del número óptimo de clusters
> plot(1:15, wss, type = "b", xlab = "Número de Clusters", ylab = "Suma de cuadrados$
+      main = "Método del codo para selección de K")
>
> # Aplicar K-Means con el número óptimo (cambiar este número según el gráfico)
> numClusters <- 6  # Ajustar según el gráfico
> set.seed(100)
> clusters <- kmeans(dataGDS[, -1, with = FALSE], centers = numClusters, nstart = 10)
>
> # Agregar los clusters al dataset
> dataGDS$Cluster <- as.factor(clusters$cluster)
>
> # Extraer solo las columnas numéricas (excluyendo identificadores y la columna de $
> data_numeric <- dataGDS[, -c(1, ncol(dataGDS)), with = FALSE]
>
> # Verificar que todas las columnas sean numéricas
> str(data_numeric)
Classes 'data.table' and 'data.frame':  18418 obs. of  25 variables:
 $ MM909_01_1: num  -0.13 -1.09 2.71 -1.09 -1.09 ...
 $ MM909_02_1: num  -0.3269 -1.0922 0.0308 -1.0922 -1.0922 ...
 $ MM909_03_1: num  -0.0654 -1.0747 1.5881 -1.0051 -1.0073 ...
 $ MM909_06_1: num  0.503 -1.096 2.239 -1.076 -1.096 ...
 $ MM909_11_1: num  -0.331 -1.094 2.947 -1.063 -1.094 ...
 $ MM909_14_1: num  -0.1391 -1.0873 0.0284 -1.0537 -1.0873 ...
 $ MM909_15_2: num  0.926 -1.08 2.517 -1.066 -1.08 ...
 $ MM909_16_1: num  0.23 -1.08 1.47 -1.06 -1.08 ...
 $ MM909_17_1: num  0.703 -1.079 1.901 -1.047 -1.049 ...
 $ MM909_22_2: num  -0.329 -1.102 3.131 -1.063 -1.102 ...
 $ MM909_25_1: num  0.464 -1.092 3.031 -1.096 -1.027 ...
 $ MM909_26_1: num  -0.413 -1.097 2.261 -1.09 -1.097 ...
 $ MM909_27_1: num  -0.28 -1.07 1.76 -1.06 -1.04 ...
 $ MM909_29_1: num  -0.682 -1.092 2.866 -1.091 -1.092 ...
 $ MM909_31_1: num  -0.311 -1.089 2.488 -1.089 -1.079 ...
 $ MM909_34_2: num  0.86 -1.075 2.666 -1.059 -0.973 ...
 $ MM909_35_2: num  -0.456 -1.0974 0.0298 -1.0974 -1.0717 ...
 $ MM909_36_1: num  0.177 -1.088 2.423 -1.088 -1.088 ...
 $ MM909_37_1: num  -0.596 -1.103 2.878 -1.103 -1.103 ...
 $ MM909_40_1: num  -0.486 -1.096 3.042 -1.09 -1.096 ...
 $ MM909_42_1: num  -0.545 -1.085 0.025 -1.089 -1.089 ...
 $ MM909_43_1: num  -0.337 -1.093 2.218 -1.07 -1.093 ...
 $ MM909_45_1: num  -0.7 -1.1 2.75 -1.1 -1.1 ...
 $ MM909_46_1: num  -0.342 -1.095 2.947 -1.09 -1.095 ...
 $ MM909_47_1: num  -0.2351 -1.0751 0.0279 -1.0803 -1.0518 ...
```

```
C:\Users\luisc\Desktop\Cosas\Universidad\3º Tercero\Segundo Cuatrismestre\Bioinformática\R\EPD6P2.R - Editor R

# Verificar estructura final
str(dataGDS)

# --- CLUSTERIZACIÓN ---
# Determinar el número óptimo de clusters con el método del codo
wss <- numeric(15)
for (i in 1:15) {
  kmeans_result <- kmeans(dataGDS[, -1, with = FALSE], centers = i, nstart = 10)
  wss[i] <- kmeans_result$tot.withinss
}

# Graficar la selección del número óptimo de clusters
plot(1:15, wss, type = "b", xlab = "Número de Clusters", ylab = "Suma de cuadrados dentro del grupo",
     main = "Método del codo para selección de K")

# Aplicar K-Means con el número óptimo (cambiar este número según el gráfico)
numClusters <- 6  # Ajustar según el gráfico
set.seed(100)
clusters <- kmeans(dataGDS[, -1, with = FALSE], centers = numClusters, nstart = 10)

# Agregar los clusters al dataset
dataGDS$Cluster <- as.factor(clusters$cluster)

# Extraer solo las columnas numéricas (excluyendo identificadores y la columna de clusters)
data_numeric <- dataGDS[, -c(1, ncol(dataGDS)), with = FALSE]

# Verificar que todas las columnas sean numéricas
str(data_numeric)

# Visualizar los clusters con PCA
fviz_cluster(clusters, data = data_numeric, geom = "point",
             ellipse = TRUE, main = "Visualización de Clusters")

# Guardar el dataset procesado con clusters
fwrite(dataGDS, "C:/Users/luisc/Desktop/RNA_Procesado.txt", sep = "\t")

# Mostrar resumen de los clusters
table(dataGDS$Cluster)
```

Incluso podemos ver el resultado de cluster.



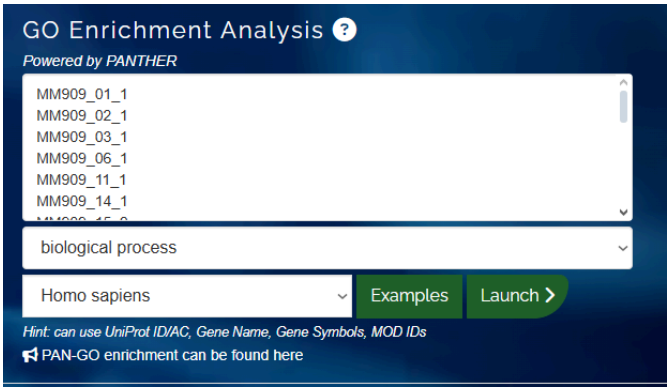
Una vez terminado el proceso deberemos tener el archivo procesado en la ruta correspondiente, en mi caso el Escritorio.



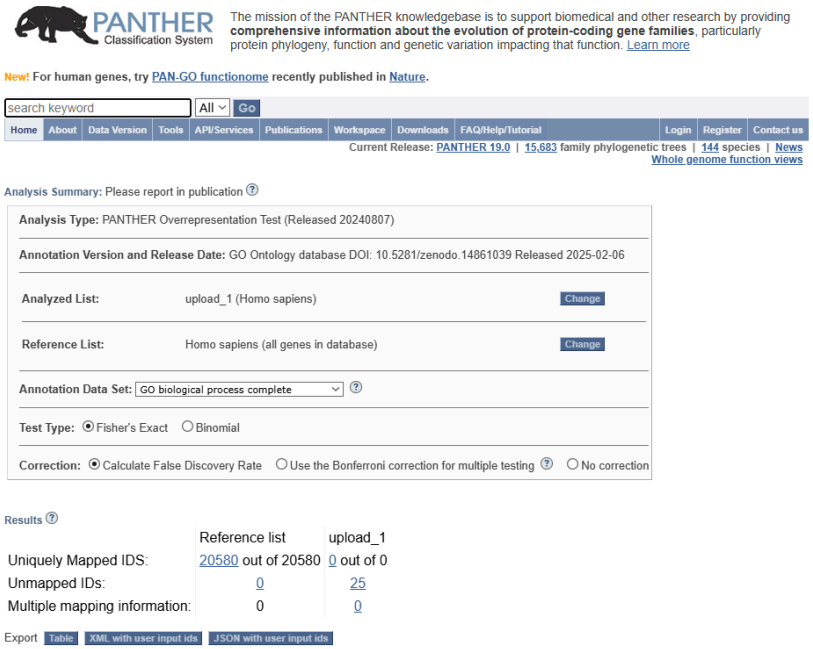
Tras esto pasaremos a hacer un análisis completo en ambas aplicaciones GeneOntology y GOrilla del dataset procesado.

GeneOntology

En primer lugar debemos subir nuestro dataset a la web de la herramienta y pulsar sobre Launch.



Tras esto se nos abrirá la siguiente pestaña, en la cual podemos ver que ningún identificador de la lista fue reconocido como mapeable.



Sin embargo podemos ver resultados más significativos que nos hará un análisis más exhaustivo de nuestro dataset.

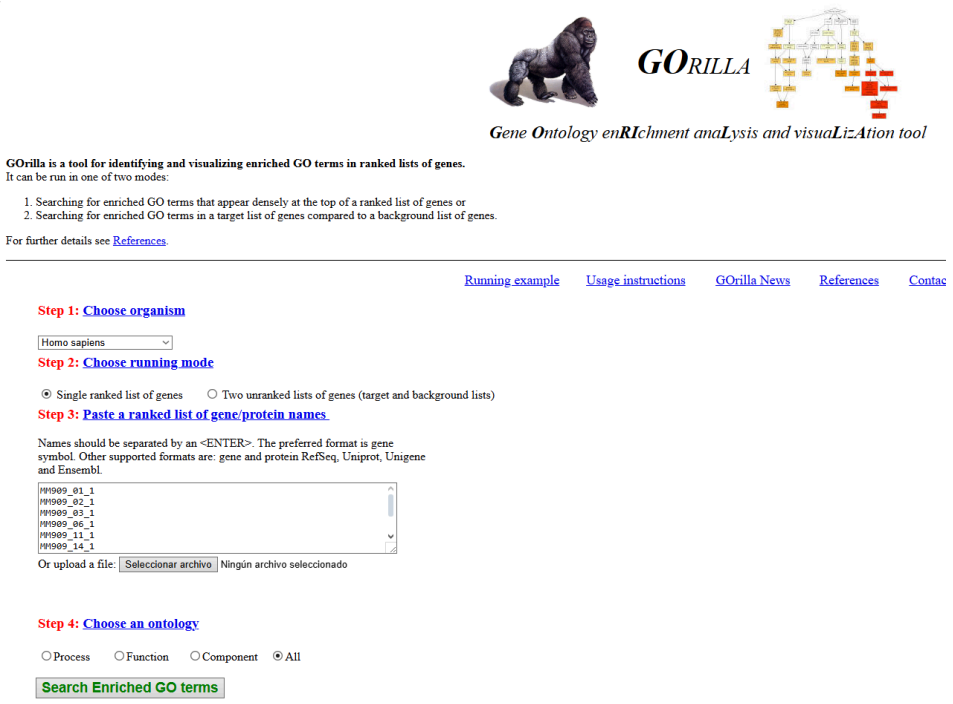
Displaying all results; click here to display only significant results

	Homo sapiens (REF)			upload_1 (▼ Hierarchy NEW!)			
GO biological process complete	#	#	expected	Fold Enrichment	+/-	raw P value	FDR
protein hexamerization	9	0	.00	-		1.00E00	1.00E00
↳protein complex oligomerization	271	0	.00	-		1.00E00	1.00E00
↳protein-containing complex assembly	1270	0	.00	-		1.00E00	1.00E00
↳cellular component assembly	2501	0	.00	-		1.00E00	1.00E00
↳cellular component organization	5872	0	.00	-		1.00E00	1.00E00
↳cellular component organization or biogenesis	6083	0	.00	-		1.00E00	1.00E00
↳cellular process	15523	0	.00	-		1.00E00	1.00E00
↳cellular component biogenesis	2746	0	.00	-		1.00E00	1.00E00
↳protein-containing complex organization	1425	0	.00	-		1.00E00	1.00E00
quinolinate catabolic process	1	0	.00	-		1.00E00	1.00E00
↳pyridine-containing compound catabolic process	55	0	.00	-		1.00E00	1.00E00
↳pyridine-containing compound metabolic process	136	0	.00	-		1.00E00	1.00E00
↳metabolic process	7621	0	.00	-		1.00E00	1.00E00
↳catabolic process	2013	0	.00	-		1.00E00	1.00E00
↳dicarboxylic acid catabolic process	17	0	.00	-		1.00E00	1.00E00
↳carboxylic acid catabolic process	225	0	.00	-		1.00E00	1.00E00
↳organic acid catabolic process	225	0	.00	-		1.00E00	1.00E00
↳organic acid metabolic process	793	0	.00	-		1.00E00	1.00E00
↳small molecule metabolic process	1575	0	.00	-		1.00E00	1.00E00
↳cellular catabolic process	297	0	.00	-		1.00E00	1.00E00
↳small molecule catabolic process	350	0	.00	-		1.00E00	1.00E00
↳carboxylic acid metabolic process	763	0	.00	-		1.00E00	1.00E00
↳oxoacid metabolic process	785	0	.00	-		1.00E00	1.00E00
↳dicarboxylic acid metabolic process	95	0	.00	-		1.00E00	1.00E00
↳quinolinate metabolic process	5	0	.00	-		1.00E00	1.00E00
positive regulation of T cell anergy	6	0	.00	-		1.00E00	1.00E00
↳positive regulation of T cell tolerance induction	10	0	.00	-		1.00E00	1.00E00
↳regulation of T cell tolerance induction	14	0	.00	-		1.00E00	1.00E00
↳regulation of tolerance induction	20	0	.00	-		1.00E00	1.00E00

En resumen, el análisis indica que, con la información y configuración actual, el dataset no muestra un enriquecimiento relevante en procesos biológicos bajo el marco de Gene Ontology.

GOrilla

Para terminar accedemos a la herramienta de GOrilla, añadimos el dataset y pulsamos sobre Searh Enriched GO terms.



Tras esto la web nos lanza lo siguiente.

No GO Enrichment Found

There are no GO terms with an enrichment p-value above the value you specified

You entered only 25 genes and chose to use a single ranked list.
If you intend to use a target versus background approach please use "Two unranked lists" option, and supply a background list.
If indeed you intended to use the ranked list approach please be aware that the list you have entered should also include the entire set o genes that go into your ranking (for example, all the genes on your array)

Note that only 0% of the terms you entered were used in the analysis

Make sure that the correct species was selected

Species used: Homo sapiens

The system has recognized 0 genes out of 25 gene terms entered by the user.
0 genes were recognized by gene symbol and 0 genes by other gene IDs .

The GOrilla database is periodically updated using the GO database and other sources.
The GOrilla databse was last updated on Mar 6, 2021

Con ello deducimos que no se encontró ningún término GO enriquecido, ya que los genes introducidos no fueron reoconocidos.